Hi everyone!

I'm going to discontinue the previous series of tutorials (tutorials 1-3), for a couple of reasons:

1. The recent Games & Play Lab session (Unity) has progressed to a level significantly past what my tutorials have reached at this point.
2. If you have been to / watched the video for the Games & Play Lab session, and / or read these tutorials, you would be at a sufficient level to understand the many, *many* Unity tutorials, on Youtube, websites, or anywhere else. These tutorials are far better than any I would be able to provide in the near future.

However, what I am going to focus on is coding. Learning how to code isn't strictly necessary to make a Unity game, but it is good to understand the basics so you can tweak code from tutorials to suit your needs.

The coding language that Unity uses is C#. If you've used C, C++, it's fairly similar.

Here are some basic concepts for coding.

SYNTAX:

C# has certain rules you have to follow about writing code. Here are a few important ones:

1. At the end of a line, you have to have a ';'. The only exceptions to this are when you are defining if statements, for/while loops, functions - basically anything with curly brackets on them.
2. If you are writing code enclosed by brackets (like in the conditions mentioned above), the code must be indented by four spaces/one tab's width.

VARIABLES:

A **Variable** is a named container used to store a value in a computer's memory. You use it to store a value, and reference it by using its name. An example is below:

```
1    public int variable_name = 5;
2    // variable is now 5.
3    variable_name = variable_name + 1;
4    // variable is now 6.
5    variable_name = 4;
6    // variable is now 4.
```

You need to initialise, or define, a variable before you start using it. The 'public' part is kind of irrelevant, and we can ignore that for now. The 'int' part is the data type. You use the following types for the following things:

Data Type	Use
Int	Whole number
Float/double (use float)	Decimal number
String	Text
Char	One text character
Bool, or Boolean	True / False value
Lists	A list of a certain data type, such as a list of ints (numbers) or strings (words). You can access specific items in a list by referring to their place in the list.

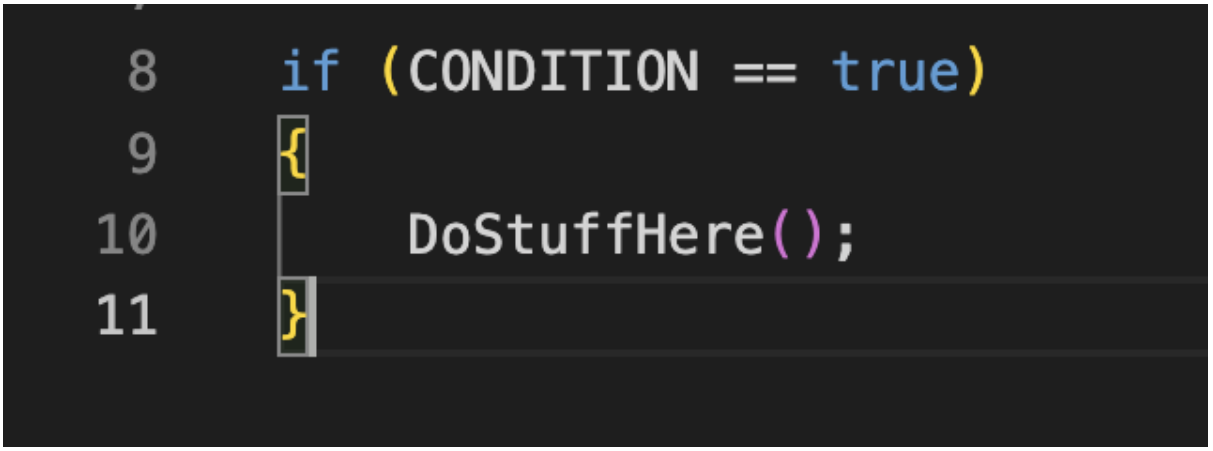
*this isn't all the data types, just some common ones

The next part, 'variable_name', is the name of the variable. You use it when you want to reference the variable. When you first name your variable, it has to be one word, and the first character must be a letter, not a number or special character. Try to make it descriptive!

Finally, the '= 5' bit is the initial value of the variable, which you need to set, unless you want it to be 'null'.

IF STATEMENTS:

An **If Statement** is a simple coding structure used in all scripting languages. Basically, it says "IF a certain condition (like x being equal to 5) is true, then execute the code within the curly brackets. Otherwise, continue on like this statement doesn't exist". To write an If statement in c#, use the following syntax:



The condition, as you can see, is written inside parentheses, and the code that is executes if the statement is true is indented and contained within curly brackets.

You can have more detail in your if statements; for instance, you can say “IF something is true do this, but IF that something else is true then do something else. This is called an “else if” or “elif” statement.

```
if (CONDITION == true)
{
    DoStuffHere();
} else if (OTHER_CONDITION == true)
{
    DoOtherStuffHere();
}
```

You can have an if statement with multiple conditions, like so:

```
if (CONDITION == true && NUMBER <= 1 || CONDITION_3 == "Dave")
{
    DoStuffHere();
}
```

The “&&” represents an ‘AND’, whereas the “||” represents an “OR”. You can also have less than, less than or equal, greater than, or greater than equal to signs for numerical values.

However, you can do a special ‘else if’ that covers all possibilities other than your previous if statements, like “IF CONDITION 1 is true do this, OTHERWISE IF CONDITION 2 is true do this other thing, BUT IF NEITHER OF THOSE IS TRUE, do this”. This is called an ‘else’ statement, and can be made like below:

```
if (CONDITION == true)
{
    DoStuffHere();
} else if (OTHER_CONDITION == true)
{
    DoOtherStuffHere();
}
else
{
    DoYetAnotherThing();
}
```

WHILE LOOPS:

While loops are another common coding structure, which basically repeats a specific block of code until a condition is no longer true. Think of it like this; your code runs, then encounters an if statement; is CONDITION true? If so, runs this code. As long as the condition is true, more if statements keep popping up, to run the code again and again until the condition is false.

```
20    while (CONDITION == true)
21    {
22        DoSomethingHere();
23    }
```

The code inside the curly brackets is going to keep running again and again and again until CONDITION is no longer true. It is entirely possible to build an open loop; a while loop that never stops. Modern coding environments, including Unity and Visual Studio, can stop that and crash the program intentionally, but it's still something to think about.

FOR LOOPS:

For loops are another type of loop that we can use. With for loops, they require an iterable, like a list, to run, because they run once for each item in a list. For loops take a variable (in the example below it's called list_item), which is supposed to represent the current item in the list. It changes which item in the list it refers to on each cycle of the loop.

```
foreach (string list_item in my_list)
{
    list_item = "Dave";
}
```

You can also make it repeat a specific number of times, like so:

```
for (int i = 0; i < 10; i++)
{
    VARIABLE = i;
}
```

This particular one repeats 10 times, and sets the variable "VARIABLE" to the current number of cycles performed each time.

Remember; anything that can be done with a for loop can be done with a while loop, and vice versa. It's just a question of which one is easier to write, or which one is more efficient.

FUNCTIONS:

If you did post-year 10 high school Math, you will remember the concept of a function. It's like a machine. You put in an input (typically x), and you get an output (typically y), and in Math, you graphed all those outputs against their inputs to form a line or curve.

Functions in programming, or C# specifically, are similar. You put in a value or values when you call the function, the function takes the inputs and runs the block of code in the function, and outputs a value or values. Functions are typically used if you have a specific block of code that you want to use in a lot of different parts of your program. You can set up and use a function as follows:

```
int AddNumberTogether(int input_1, int input_2)
{
    output = input_1 + input_2;
    return output;
}

this_var_will_be_five = AddNumberTogether(2,3);
```

The “int” at the start of the first line is the data type that the output is going to be. The function name, in this case “AddNumberTogether”, is next. Then, inside the parentheses, we have the inputs and their data types. ‘Input_1’ and ‘input_2’ are the two variables that are going to be used to refer to the input values. The block of code inside the curly brackets performs an operation (in this case, adding the two inputs), then *returns* the output. The value stored in the variable next to that return keyword will be the value that is provided as output by the function, in the last line.

Unity has many in-built functions, most notably “Start” and “Update” which are called at the game’s initialisation and every frame, respectively. They have many more functions that are called at specific occurrences, like an object making contact with another’s collider. Learn about them and make use of them.

That’s all from me for now, I gotta lock in on my exams and such. Good luck everyone!